

Two-Mesh Grid Refinement Method

- curvilinear quasi-orthogonal 2D mesh (magnetic flux surface structure) as a basis grid (left side).
- geometrical values stored in 2D matrices
- numerical grid is a rough, flexible grid
- physical values stored in 1D lists

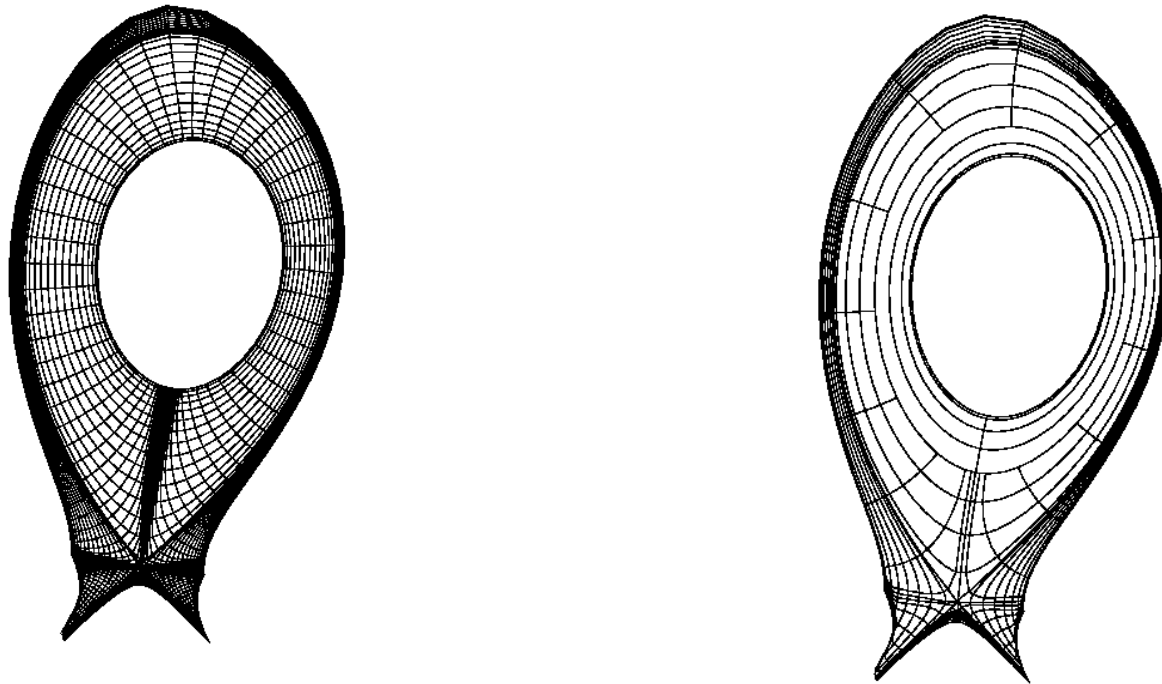


Figure: Basis grid (128x32) and numerical grid with core cells

Preparation (in SOLPS5.0)

As a first step towards grid refinement/coarsening the B2-code has been transferred to a non-staggered grid and the correction scheme due to Rhie and Chow has been added.

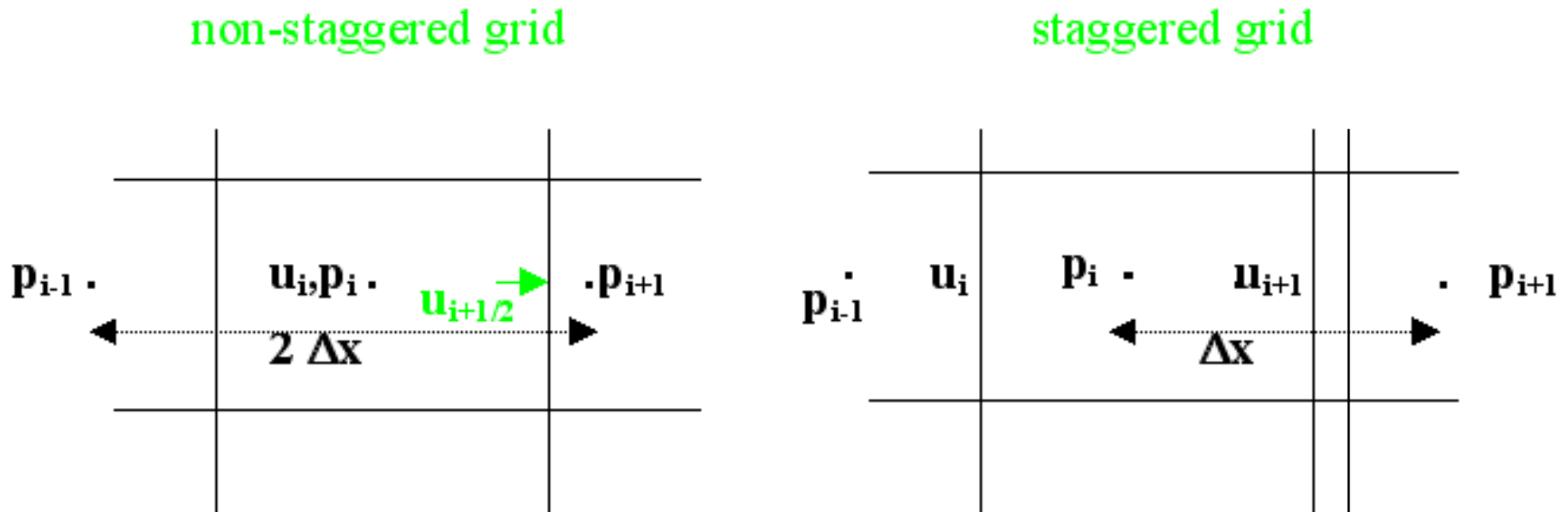


Figure: Comparison of non-staggered and staggered grids

New files, etc...

- New source directory for grid adaptation routines:
\$SOLPSTOP/src/Braams/b2/src/grid
- New modules: **b2mod_indirect**, **b2mod_geo2**
- New variables within **b2fstatx** files
- New converter program **b2cv** for **SOLPS5.0**
- Self-consistent use of dimensioning variables
- File versions for all input/output files (default: 02.001.000)
- Full backward-compatibility with **SOLPS5.0**:
 - Same physics
 - Same switches and input file structure
 - Drifts are computed from basis mesh magnetic field information

New switches

- All documented in **b2cdci**
- Grid adaptation model '**b2grmn_grk**'
- Grid adaptation frequency '**b2grmn_ngr**'
- Fusion criteria '**b2grmn_fa(:)**', division '**b2grmn_da(:)**'
- '**b2plas_iaverage**': averaging model for fusing cells
- '**b2grbc_neutrals_namelist**', '**b2grbc_boundary_namelist**'
 - cell numbers correspond to basis mesh
 - cells can belong to only one boundary

* `b2grmn_grk'` `0'` Run

* Type of grid refinement/coarsening criteria. If `grk.eq.0`, no grid changes are made. If `grk.eq.1` or `grk.eq.3`, the `da()` and `fa()` parameters are used as below. `grk.eq.2` is reserved for debugging purposes. See description of `fa()` and `da()` below for more details. When `grk.eq.1`, all cell fusion and/or division conditions on all plasma parameters and species are checked. The option `grk.eq.3` is left for the user to define his own selection of criteria (by adding or removing appropriate elements in the `b2grmn` routine).

* `b2grmn_da(1)'` `1.0'` Run

* `b2grmn_da(2)'` `1.0'` Run

* `b2grmn_da(6)'` `1.0'` Run

* `b2grmn_da(8)'` `1.0'` Run

* `b2grmn_fa(1)'` `0.0'` Run

* `b2grmn_fa(2)'` `0.0'` Run

* `b2grmn_fa(3)'` `0.0'` Run

* `b2grmn_fa(6)'` `0.0'` Run

* `b2grmn_fa(7)'` `0.0'` Run

* Cell division (`da`) and cell fusion (`fa`) parameters. The indices (1), (2) and (3) apply to the temperature, density and potential criteria, while indices (6) through (8) apply to the velocity criteria. The following is valid only for `grk.eq.1` or `grk.eq.3`. The arrays `fa()` and `da()` are dimensioned of length (10), but the other elements are currently not used.

* The relevant cell division conditions are as follows:

* For Te, Ti, Na and Po, if two neighbouring cells have normalized gradients greater than `da(1)` or there is a normalized gradient larger than `da(2)` between their perpendicular neighbours, then the cell should be divided.

* For Ua, if two neighbouring cells have normalized gradients greater than `da(6)`, then the cell should be divided. Additionally, if the difference in Ua between neighbouring cells is larger than `da(8)`, then the cell should be divided. However, all Ua-based division criteria can be turned off if `da(6).ge.10`.

* The relevant cell fusion conditions are as follows:

* For Te, Ti, Na and Po, if two neighbouring cells have normalized gradients less than `fa(1)` and their fusion would not cause a normalized gradient larger than `fa(2)`, they may fuse. Additionally, there must be normalized gradient smaller than `fa(3)` between their perpendicular neighbours.

* For Ua, if two neighbouring cells have normalized gradients less than `fa(6)` and their fusion would not cause a normalized gradient larger than `fa(7)`, they may fuse.

* In all the above, by normalized gradient we mean : $\frac{\text{abs}(X(i)-X(j))}{\min(\text{abs}(X(i),X(j)))}$ where X is the quantity being tested, i is the cell index of the cell being tested, and j is the cell index of the neighbour(s) to cell (i).

Variables for basic mesh

in `b2mod_geo2`

- coordinates: `crx1(-1:nx1,-1:ny1,0:3),cry1`
- cell variables: `vol1,hx1,hy1`
- others: `bb1,ffbz1,fpsi1,qz1`
- neighbors: `leftix1,leftiy1,....`
- cuts: `leftcut1,rightcut1,...`
- miscellaneous: `region(:,,0:2),...`

Variables for physical mesh

in **b2mod_indirect.F**

- Note arrays are defined of length **nn**, but there are only **n** active cells
- geometrical information:
 - **ixl(nn), ixh, iyl, iyh** borders for basic cells
 - coarse cell **i** covers basis cells (**ixl(i), iyl(i)**) to (**ixh(i),iyh(i)**)
 - **hx, hy, vol, crx, cry** coordinates,distances
 - constructed from basis cell geometry information
- others:
 - **bb, ffbz,...**
- Neighborhood relations:
 - **left(nn), leftn(nn,nmax), right,...** Neighbor relations
 - Cell **i** has **left(i)** left neighbors, with indices **leftn(i,1:left(i))**
 - If **left(i).eq.0**, then cell **i** is a “Western” edge cell
 - **region1(nn,0:2)** region index

Variables for physical mesh in **b2mod_indirect.F** (2)

- **kindc(i,1):**
 - 0: not defined as cell
 - 1: normal physical cell
 - 2: 1-D core cell
 - 3: boundary cell
- **kindc(i,2):**
 - 0: ordinary cell
 - 1-4: “cut” on left/right/top/bottom edge of cell
 - 5-6: “cuts” on both right and top/bottom edges
 - 7-9: “cuts” on both left and top/bottom/right edges
 - 10-11: “cuts” on left and right edges and top/bottom edge
 - 12: “cuts” on top and bottom edges
 - 13-14: “cuts” on top and bottom edges and left/right edge
 - 15: “cuts” on all four edges
- **kindc(i,3):** boundary index from **b2.boundary.parameters**
- **kindc(i,4):** stratum index from **b2.neutrals.parameters**

Physical variables in **b2mod_plasma.F**

physics: $te(nn)$, $ti(nn)$, $na(nn,ns)$, $ua(nn,ns)$, $po(nn)$

fluxes:

$fhe(nn, nmax, 0:3)$, $fna(nn, nmax, 0:3, ns), \dots$

- 1-D lists for plasma state variables
- **nmax** is set in **b2ag.dat** (modified by **b2yt**)
- fluxes computed in all 4 directions, 2 obtained by symmetry (0: left, 1: bottom, 2: right, 3: top)
- starting physical mesh is given by **nx** $\times$ **ny** in **b2ag.dat**
- fluxes are located at the interface between a cell and each of its neighbours

Grid Refinement

When: every **ngr** time steps, call to **b2grmn** from **b2mndt**
 Refinement **Criteria for Fusion** (example in x-direction):

- Necessary
- entire interface is shared
 - no “cut” in between
 - same region
 - not more than **nmax** neighbours
 - same boundary (stratum) status
- b2gcfx:** User sets F_1, F_2, F_3 :
- $$|T - T_{right}| < F_1 \text{Min}(T, T_{right})$$
- AND
- $$\text{Max}(|T - T_{allleft}|) < F_2 \text{Min}(T, T_{right})$$
- AND
- $$\text{Max}(|T_{right} - T_{allright}|) < F_2 \text{Min}(T_{allright}, T_{right})$$
- AND
- b2gcfxy:**
- $$\text{Max}(|T_{above} - T_{above,all}|) < F_3 T_{above,1}$$

Refinement Criteria for Division

example in x-direction

Necessary:

- physical cell consists of more than 1 basic cell
- not creating more than **nmax** neighbors
- The cell is divided:
 - at its mid-point if the parallel gradient is too strong
 - at the interface between top or bottom neighbors for radial conditions

$$\mathbf{b2gcdx}: \quad \text{Max}(|T - T_{allleft}|) > D_1 T \quad \text{OR} \quad \text{Max}(|T - T_{allright}|) > D_1 T$$

OR

User sets **D₁** and **D₂**:

$$\mathbf{b2gcdxy}: \quad \text{Max}(|T_{top} - T_{alltop}|) > D_2 T_{top,1}$$

For all physical variables: **te**, **ti**, **na(ns)**, **ua(ns)**, **po**
factors in **b2mn.dat** !!!!

Fused / Divided plasma quantities

- 3 models for fusion controlled by **'b2plas_iaverage'**
 - 1: arithmetic average
 - 2: volume average
 - 3: physics-based average
 - fluxes to common neighbors are summed
- Division:
 - radial: just copy over quantities
 - parallel: interpolate gradient (not to break parallel flow)
 - fluxes are distributed according to area of contact

Comparison of hard and weak conditions

without **ua**:

- good grid fusion in center (4420->2456)
- but physical results different especially **ua**

with full **ua**:

- grid fusion in center disturbed (4420->3464)
- physical results exactly as without grid refinement

no **ua** fusion condition:

- good grid fusion in center (4420->2512)
- physical results similar to “real” case

b2agdr: **b2agmt:** calculation of metric coefficients for basis mesh
b2gnew: define starting mesh
b2gmvc: calculation of metric coefficients for start mesh

b2aidr: **b2wngm2**
b2wups2

b2fstati: **crx, fpsi, bb, left, leftn,...**
na, ua, te,...

b2mndr:

itim=0: b2grbc
(modify grid because
of bound. cond.)

all **ngr** steps:
grid refinement-
coarsening in
b2grmn

graphics output:
plot2gr,
plot2ne

b2grmn:

n>nn OR
n<nn/3

ref-coarse cond.:
grk, da(:),fa(:)
b2gcfx,b2gcfxy,
b2gcfy,b2gcfyx,
b2gcdx,...

b2grnr: min. n
b2gmvc: hx,...
b2grts: tests

Allocation Module:
write **b2fstate**
DEALLOCATE
ALLOCATE
b2gral: read
b2fstate

ref-coarse made:
• **b2grxf**
• **b2grxd**
• **b2gryf**
• **b2gryd**

NUMERICS

Automatic Matrix allocation

This feature optimizes the used matrix dimensions automatically to save workspace.

n>nn: new matrix dimension **nf**×**nn**

n<nn/3: new matrix dimension **nn/2**

Matrix Solver

Using a 5-point-solver every cell variable is influenced by up to 4 **nmax** neighboring cell values.

$$\sum_{l=\text{bottom}} a_{k,l} n_l + \sum_{l=\text{left}} b_{k,l} n_l + c_k n_k + \sum_{l=\text{right}} d_{k,l} n_l + \sum_{l=\text{top}} e_{k,l} n_l = r_k$$

This results in a matrix structure **An=r** with a square sparse 2D-matrix **A** solved by standard matrix-solvers.

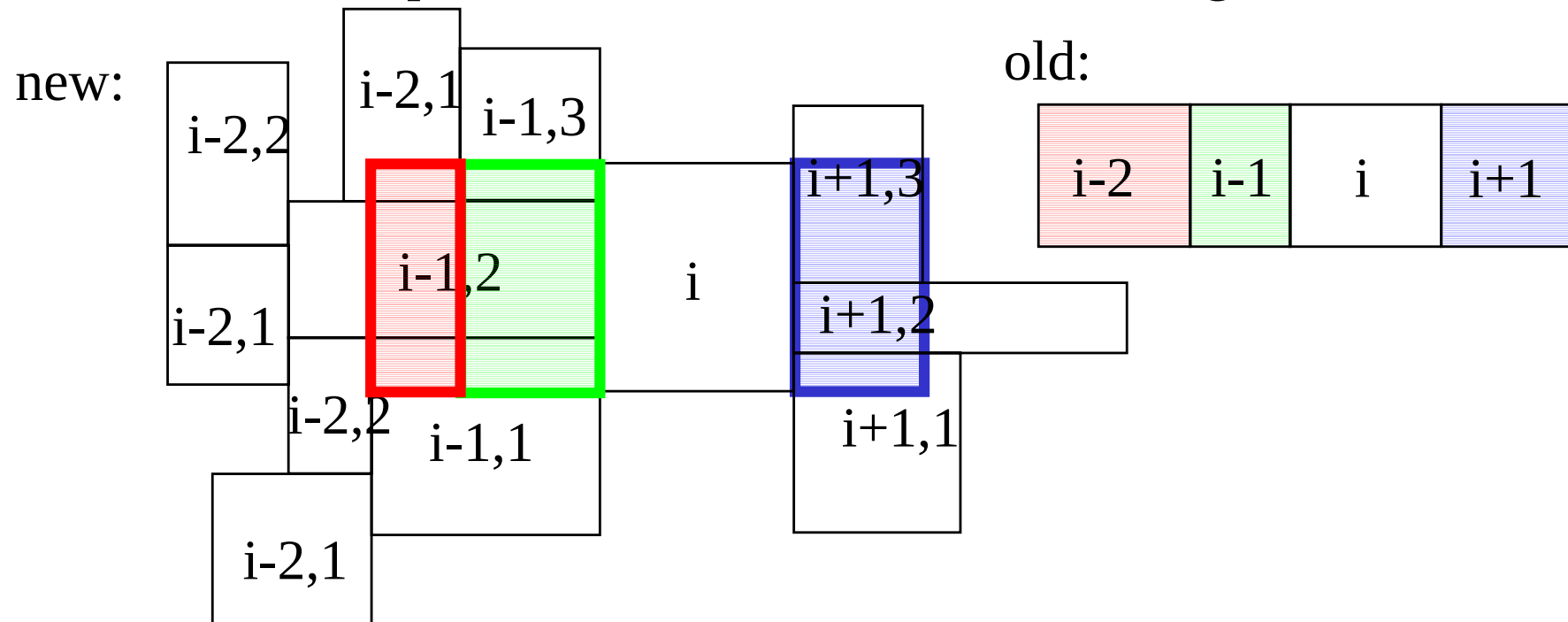
Of previous SOLPS5.0 solvers, only **MA28** still works!

Interpolation scheme

- volume interpolation not valid any more
- **hx** and **hy** instead of volumes, standardized scheme
- up to **nmax** neighbors
- interpolation to cell faces also weighted with connecting areas

Method of Rhie and Chow

- in **b2tfnb**: pressure influence of left of left neighbours,...



New b2plot features

- All variables mapped back to basis mesh
- **f.x** and **f.y** show flat steps where the cells are merged
- **sm.x** and **sm.y** provide smoothed profiles
- **bmsh** show the basis mesh, while **mesh** show the working mesh
- **bgrd** overlays the basis mesh, while **grid** overlays the working grid
- **kind** gives all 4 pieces of the **kindc** array